



МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Новосибирский государственный технический университет»



**НГТУ
НЭТИ**

**Факультет прикладной
математики и информатики**

Кафедра теоретической и прикладной информатики
Практическое задание № 1
по дисциплине «УРАВНЕНИЯ МАТЕМАТИЧЕСКОЙ ФИЗИКИ»

РЕШЕНИЕ ЭЛЛИПТИЧЕСКИХ КРАЕВЫХ ЗАДАЧ МЕТОДОМ КОНЕЧНЫХ РАЗНОСТЕЙ

Вариант 3

ПМ-34 РУБЦОВ АРТЁМ

ПМ-34 КУДЮКОВ МАКСИМ

Преподаватели

ЗАДОРОВНИЙ АЛЕКСАНДР ГЕННАДЬЕВИЧ

ЛЕОНОВИЧ ДАРЬЯНА АЛЕКСАНДРОВНА

Новосибирск, 2026

Содержание	
1 Цель работы	3
2 Задание	3
3 Теоретическая часть.....	3
4 Исследования.....	4
4.1 Линейная функция	4
4.2 Исследование для разных сеток и коэффициентов	4
5 Выводы	5
6 Текст программы.....	2
6.1 Main.cpp	2
6.2 Problem.h	2
6.3 Problem.cpp	3
6.4 Element3D.h	15
6.5 Element3D.cpp.....	16
6.6 Basis3D.h	18
6.7 Basis3D.cpp	18
6.8 Quadratures.h	20
6.9 Quadratures.cpp	21
6.10 Mesh3D.h.....	21
6.11 Mesh3D.cpp	22
6.12 SparseMatrix.h	24
6.13 SLE.h.....	24
6.14 SLE.cpp.....	24
6.15 SLEAssistant.h.....	26
6.16 SLEAssistant.cpp	26
6.17 LU.h	28
6.18 LU.cpp.....	28
6.19 MSG.h	30
6.20 MSG.cpp	30
6.21 Func.h.....	35
6.22 Func.cpp.....	35

1 Цель работы

Разработать программу решения гармонической задачи методом конечных элементов. Провести сравнение прямого и итерационного методов решения получаемой в результате конечноэлементной аппроксимации СЛАУ.

2 Задание

Решить трехмерную гармоническую задачу в декартовых координатах, базисные функции – трилинейные.

3 Теоретическая часть

Изначально имеем гиперболическую задачу:

$$\chi \frac{\partial^2 u}{\partial t^2} + \sigma \frac{\partial u}{\partial t} - \operatorname{div}(\lambda \operatorname{grad} u) = f$$

В нашем случае функции u и f представимы в виде:

$$u(x, y, z, t) = u^s(x, y, z) \sin \omega t + u^c(x, y, z) \cos \omega t$$

$$f(x, y, z, t) = f^s(x, y, z) \sin \omega t + f^c(x, y, z) \cos \omega t$$

Отсюда, подставляя u и f в гиперболическое уравнение и приравнивая слагаемые при синусах и косинусах в левой и правой частях, получаем систему уравнений:

$$\begin{cases} -\operatorname{div}(\lambda \operatorname{grad} u^s) - \omega \sigma u^c - \omega^2 \chi u^s = f^s \\ -\operatorname{div}(\lambda \operatorname{grad} u^c) + \omega \sigma u^s - \omega^2 \chi u^c = f^c \end{cases}$$

С краевыми условиями:

$$u^s(S_1) = u_g^s,$$

$$u^c(S_1) = u_g^c$$

После конечноэлементной аппроксимации имеем:

$$p_{ij} = G_{ij} - \omega^2 \chi M_{ij}$$

$$c_{ij} = \omega \sigma M_{ij}$$

Матрица конечноэлементной СЛАУ будет иметь следующую структуру:

$$A = \begin{pmatrix} p_{11} & -c_{11} & p_{12} & -c_{12} & & p_{1n} & -c_{1n} \\ c_{11} & p_{11} & c_{12} & p_{12} & & c_{1n} & p_{1n} \\ p_{21} & -c_{21} & p_{22} & -c_{22} & & p_{2n} & -c_{2n} \\ c_{21} & p_{21} & c_{22} & p_{22} & & c_{2n} & p_{2n} \\ & & & & \dots & & \\ & & & & & \dots & \\ p_{n1} & -c_{n1} & p_{n2} & -c_{n2} & & p_{nn} & -c_{nn} \\ c_{n1} & p_{n1} & c_{n2} & p_{n2} & & c_{nn} & p_{nn} \end{pmatrix}$$

Погрешность решения $u - u^*$ будем считать в норме L_2 как от непрерывных функций в момент времени $t = 1$. В качестве итерационного метода использовался MSG с диагональным предобуславливанием, а в качестве прямого метода – LU.

4 Исследования

u^* - истинное решение, u – численное решение, $\|u^* - u\|_{L2}$ – норма в пространстве L_2 , краевые условия везде первого рода.

4.1 Линейная функция

$u^s = -0.15 + 0.35x + 0.45y - 0.25z;$ $u^c = 0.3 - 0.2x + 0.15y + 0.5z;$ $\lambda = 2; \sigma = 1; \omega = 4; \chi = 3.$
$ u^s - u^c _{L2} = 8.535498e-13$ $ u^c - u _{L2} = 1.651399e-12$ $ u^* - u _{L2} = 1.010087e-12$

4.2 Исследование для разных сеток и коэффициентов

$$u^s = 0.35x^2 + 0.45y^2 - 0.25z^2;$$

$$u^c = 0.2x^2 + 0.15y^2 + 0.5z^2;$$

Количество узлов-729

λ	ω	σ	χ	время MSG, мс	$\ u - u^*\ _{MSG}$	время LU, мс	$\ u - u^*\ _{LU}$
100	1000	1	1.00E-11	3	1.908713e-02	1	4.489125e-02
1000				2	2.091558e-02	1	4.548884e-02
10000				3	2.093167e-02	2	4.535073e-02
100000				2	2.093141e-02	1	4.533491e-02
800000				2	2.093136e-02	1	4.533335e-02
1000	0.0001	1	1.00E-11	3	1.201763e-02	1	3.565984e-02
	0.01			3	1.218671e-02	1	3.589586e-02
	1			2	2.091623e-02	2	4.513524e-02
	100			2	1.172779e-02	1	2.858216e-02
	1.00E+04			2	2.994409e-04	1	1.548713e-03
	1.00E+06			889	1.867608e+13	1	5.964635e-04
	1.00E+09			867	4.445828e-02	1	1.763706e-03
1000	1000	0	1.00E-11	2	2.093136e-02	2	4.855577e+05
		100		3	4.099281e-03	1	2.301704e-02
		10000		864	2.208344e-02	1	1.816793e-03
		1E+06		874	1.995901e+05	1	1.843373e-03
		1E+08		848	1.037286e+03	1	1.843371e-03
1000	1000	1	8.81E-12	2	2.091558e-02	2	4.548884e-02
			1.00E-12	3	2.091558e-02	1	4.548884e-02
			1.00E-11	3	2.091558e-02	1	4.548884e-02
			1.00E-10	2	2.091558e-02	1	4.548884e-02

На данном тесте MSG с диагональным предобуславливанием не сошёлся в выделенных тестах. Поэтому на этих тестах LU лучше, так как он тратит меньше времени на решение и в некоторых тестах лучше по погрешности. На остальных комбинациях MSG сошелся и по затраченному времени примерно совпадает с LU.

Количество узлов-32768							
λ	ω	σ	χ	время MSG, мс	$\ u - u^*\ MSG$	время LU, мс	$\ u - u^*\ LU$
100	1000	1	1.00E-11	350	1.791616e-02	81	1.934643e-01
1000				355	1.981533e-02	76	1.935479e-01
10000				346	1.983213e-02	74	1.935479e-01
100000				327	1.983186e-02	75	1.935482e-01
800000				346	1.983181e-02	78	1.935482e-01
1000	0.0001	1	1.00E-11	301	1.129708e-02	74	1.523270e-01
	0.01			316	1.145789e-02	77	1.529516e-01
	1			330	1.982032e-02	75	1.935253e-01
	100			340	1.420673e-03	80	1.297164e-01
	1.00E+04			340	1.062920e-02	78	1.703599e-01
	1.00E+06			1088	6.434583e-04	77	9.870804e-02
	1.00E+09			4001	4.445828e-02	76	1.162149e-01
1000	1000	0	1.00E-11	252	1.983180e-02	75	1.935483e-01
		100		359	2.606795e-03	75	1.882967e-01
		10000		1038	1.256222e-04	77	4.419564e-03
		1E+06		4019	1.461765e-03	77	1.217822e-04
		1E+08		3987	1.198798e-02	76	1.227635e-04
1000	1000	1	8.81E-12	320	1.981533e-02	77	1.935444e-01
			1.00E-12	337	1.981533e-02	75	1.935444e-01
			1.00E-11	346	1.981533e-02	77	1.935444e-01
			1.00E-10	345	1.981533e-02	74	1.935444e-01

На данном тесте МСГ с диагональным предобуславливанием не сошёлся в выделенных тестах. Поэтому на этих тестах LU лучше, так как он тратит намного меньше времени на решение и в некоторых тестах лучше по погрешности. На остальных комбинациях МСГ сошелся и по затраченному времени хуже чем LU. Однако в тестах, в которых МСГ сошелся, погрешность меньше минимум на 1 порядок.

5 Выводы

Из проведенных исследований можно сделать вывод, что на сетках с небольшим количеством узлов выигрывает метод LU-разложения, так как МСГ с диагональным предобуславливанием не всегда сходится. На сетках с большим количеством узлов LU выигрывает по времени, но погрешность у него больше на порядок, чем у МСГ.

Также видно, что коэффициенты уравнения в основном влияют на сходимость МСГ. С изменением сетки изменяется время решения СЛАУ, погрешность у LU и сходимость у МСГ.

6 Текст программы

6.1 Main.cpp

```
#include "Problem.h"
#include "LU.h"
#include "MSG.h"
#include "SLE.h"
#include "SLEAssistant.h"
#include <algorithm>
#include <cmath>
#include <fstream>
#include <iostream>
#include <vector>

int main( )
{
    int testCase = 1;
    std::ifstream testFile( "test.txt" );
    if ( testFile )
    {
        testFile >> testCase;
    }

    switch ( testCase )
    {
    case 1:
        std::cout << "Running test case 1: 1D manufactured test\n";
        break;
    case 2:
        std::cout << "Running test case 2: 3D harmonic linear manufactured test #1\n";
        break;
    case 3:
        std::cout << "Running test case 3: 3D harmonic linear manufactured test #2\n";
        break;
    default:
        std::cout << "Unknown test in test.txt, fallback to case 1\n";
        testCase = 1;
        break;
    }

    Problem problem;
    problem.Run( testCase );
    return 0;
}
```

6.2 Problem.h

```
#pragma once
#include "Element.h"
#include "Element3D.h"
#include "Mesh3D.h"
#include "SparseMatrix.h"
#include "Mesh.h"
#include <string>
#include <vector>
class Problem
{
public:
    enum class Dimension
    {
        OneD = 1,
        ThreeD = 3
    };
};
```

```

void Run( int testCase );
const std::vector<double> &GetSolution( ) const { return q; }

private:
    struct L2Errors3D
    {
        double us = 0.0;
        double uc = 0.0;
        double u = 0.0;
    };

    enum class SolveMethod
    {
        LU,
        MSGDiagonal
    };

    Func func;

    std::vector<double> q;

    int n = 0;
    SparseMatrix matrix;

    std::vector<double> b;
    std::vector<double> boundaryB;
    std::vector<char> fixedDofs;
    SolveMethod solveMethod = SolveMethod::LU;
    Dimension dimension = Dimension::OneD;

    int elemCount = 0;
    std::vector<Element> elements;
    std::vector<Element3D> elements3D;
    std::vector<double> q3d;
    std::vector<double> usExact3D;
    std::vector<double> ucExact3D;
    int current3DTestNumber = 2;

    void Input( );
    void BuildMatrix( );
    void BuildB( );
    void Solve( );
    void ApplyDirichlet( int dof, double value );
    void PrintResult( ) const;

    void Input3D( int testNumber );
    void BuildMatrix3D( );
    void BuildB3D( );
    void ApplyDirichlet3D( int dof, double value );
    void Solve3D( );
    void PrintResult3D( );
    void Run3DHarmonicTest( int testNumber );
    L2Errors3D ComputeL2Errors3D( double t ) const;

    Mesh mesh;
    Mesh3D mesh3d;
};

```

6.3 Problem.cpp

```

#include "Problem.h"
#include "Basis3D.h"
#include "Quadratures.h"
#include "LU.h"
#include "MSG.h"
#include "SLE.h"
#include "SLEAssistant.h"
#include <algorithm>
#include <array>
#include <cctype>

```

```

#include <cmath>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <set>
#include <string>
#include <stdexcept>
#include <chrono>

void Problem::Run( int testCase )
{
    if ( testCase == 1 )
    {
        dimension = Dimension::OneD;
        Input( );
        BuildMatrix( );
        BuildB( );
        Solve( );
        PrintResult( );
        return;
    }

    if ( testCase >= 2 )
    {
        dimension = Dimension::ThreeD;
        Run3DHarmonicTest( testCase );
        return;
    }

    throw std::runtime_error( "Unknown test case. Use 1 (1D) or >=2 (3D harmonic tests)." );
}

void Problem::Input( )
{
    mesh.InputMeshes( );
    func.SetDomain( mesh.meshX.front( ), mesh.meshX.back( ) );

    std::ifstream solverFile( "solver.txt" );
    if ( solverFile )
    {
        std::string mode;
        solverFile >> mode;
        std::transform( mode.begin( ), mode.end( ), mode.begin( ), []( unsigned char c ) {
return static_cast<char>( std::toupper( c ) ); } );
        if ( mode == "MSG" )
        {
            solveMethod = SolveMethod::MSGDiagonal;
        }
        else
        {
            solveMethod = SolveMethod::LU;
        }
    }

    elemCount = mesh.n_x - 1;
    n = 2 * mesh.n_x;

    matrix = SparseMatrix( n );
    b.assign( n, 0.0 );
    q.assign( n, 0.0 );
    elements.resize( elemCount );

    std::vector<double> coords( 2, 0.0 );
    std::vector<double> localFs( 2, 0.0 );
    std::vector<double> localFc( 2, 0.0 );
    for ( int i = 0; i < elemCount; ++i )
    {
        coords = { mesh.meshX[i], mesh.meshX[i + 1] };
    }
}

```



```

        for ( int j = 0; j < 2; ++j )
        {
            localFs[j] = func.fs( mesh.meshX[i + j] );
            localFc[j] = func.fc( mesh.meshX[i + j] );
        }

        elements[i] = Element( coords, localFs, localFc );
    }
}

void Problem::BuildMatrix( )
{
    std::vector<std::set<int>> rowProfile( n );
    for ( int e = 0; e < elemCount; ++e )
    {
        const std::vector<int> map = { 2 * e, 2 * e + 1, 2 * e + 2, 2 * e + 3 };
        for ( int i = 0; i < 4; ++i )
        {
            for ( int j = 0; j < 4; ++j )
            {
                if ( map[j] < map[i] )
                {
                    rowProfile[map[i]].insert( map[j] );
                }
            }
        }
    }

    matrix.ig.assign( n + 1, 0 );
    for ( int i = 0; i < n; ++i )
    {
        matrix.ig[i + 1] = matrix.ig[i] + static_cast<int>( rowProfile[i].size( ) );
    }

    const int nnz = matrix.ig[n];
    matrix.jg.assign( nnz, 0 );
    matrix.ggl.assign( nnz, 0.0 );
    matrix.ggu.assign( nnz, 0.0 );
    matrix.di.assign( n, 0.0 );

    for ( int i = 0; i < n; ++i )
    {
        int k = matrix.ig[i];
        for ( const int col : rowProfile[i] )
        {
            matrix.jg[k++] = col;
        }
    }

    auto addToGlobal = [this]( int row, int col, double value )
    {
        if ( row == col )
        {
            matrix.di[row] += value;
            return;
        }

        if ( row > col )
        {
            for ( int k = matrix.ig[row]; k < matrix.ig[row + 1]; ++k )
            {
                if ( matrix.jg[k] == col )
                {
                    matrix.ggl[k] += value;
                    return;
                }
            }
        }
        else

```

```

        {
            for ( int k = matrix.ig[col]; k < matrix.ig[col + 1]; ++k )
            {
                if ( matrix.jg[k] == row )
                {
                    matrix.ggu[k] += value;
                    return;
                }
            }
        }

        throw std::runtime_error( "Sparse profile mismatch while assembling global matrix"
);
    };

    for ( int e = 0; e < elemCount; ++e )
    {
        Element &element = elements[e];
        element.localMatrix.assign( 4, std::vector<double>( 4, 0.0 ) );
        element.BuildLocalMatrix( );

        const std::vector<int> map = { 2 * e, 2 * e + 1, 2 * e + 2, 2 * e + 3 };
        for ( int i = 0; i < 4; ++i )
        {
            for ( int j = 0; j < 4; ++j )
            {
                addToGlobal( map[i], map[j], element.localMatrix[i][j] );
            }
        }
    }
}

void Problem::BuildB( )
{
    std::fill( b.begin( ), b.end( ), 0.0 );
    for ( int e = 0; e < elemCount; ++e )
    {
        Element &element = elements[e];
        element.localB.assign( 4, 0.0 );
        element.BuildLocalB( );

        const std::vector<int> map = { 2 * e, 2 * e + 1, 2 * e + 2, 2 * e + 3 };
        for ( int i = 0; i < 4; ++i )
        {
            b[map[i]] += element.localB[i];
        }
    }
}

void Problem::ApplyDirichlet( int dof, double value )
{
    for ( int row = dof + 1; row < n; ++row )
    {
        for ( int k = matrix.ig[row]; k < matrix.ig[row + 1]; ++k )
        {
            if ( matrix.jg[k] == dof )
            {
                if ( fixedDofs.empty( ) || !fixedDofs[row] )
                {
                    b[row] -= matrix.ggl[k] * value;
                }
                matrix.ggl[k] = 0.0;
                matrix.ggu[k] = 0.0;
                break;
            }
        }
    }

    for ( int k = matrix.ig[dof]; k < matrix.ig[dof + 1]; ++k )

```

```

{
    const int col = matrix.jg[k];
    if ( fixedDofs.empty() || !fixedDofs[col] )
    {
        b[col] -= matrix.ggu[k] * value;
    }
    matrix.ggu[k] = 0.0;
    matrix.ggl[k] = 0.0;
}

matrix.di[dof] = 1.0;
b[dof] = value;
if ( !fixedDofs.empty() )
{
    fixedDofs[dof] = 1;
}
}

void Problem::Solve()
{
    fixedDofs.assign( n, 0 );
    const int leftNode = 0;
    const int rightNode = mesh.n_x - 1;
    const double dAs_dz_right = func.thetaS( mesh.meshX[rightNode] );
    const double dAc_dz_right = func.thetaC( mesh.meshX[rightNode] );

    // Условие Неймана на правой границе.
    b[2 * rightNode] += dAs_dz_right;
    b[2 * rightNode + 1] += dAc_dz_right;

    // Условие Дирихле на левой границе.
    ApplyDirichlet( 2 * leftNode, func.boundaryValue( 0, mesh.meshX[leftNode] ) );
    ApplyDirichlet( 2 * leftNode + 1, func.boundaryValue( 1, mesh.meshX[leftNode] ) );

    auto start = std::chrono::steady_clock::now();
    if ( solveMethod == SolveMethod::MSGDiagonal )
    {
        SLE sle;
        sle.matrix = matrix;
        sle.f = b;
        sle.x.assign( n, 0.0 );
        sle.maxIter = 10000;
        sle.eps = 1e-12;
        q = MSG::SolutionWithDiagonalConditioning( sle );
        std::cout << "Solver: MSG (diagonal preconditioning)\n";
    }
    else
    {
        LU lu( matrix );
        std::vector<double> y( n, 0.0 );
        q.assign( n, 0.0 );
        SLEAssistant::LUDirect( lu, b, y );
        SLEAssistant::LUReverse( lu, y, q );
        std::cout << "Solver: LU\n";
    }
    auto end = std::chrono::steady_clock::now();
    auto duration = std::chrono::duration_cast< std::chrono::milliseconds >( end - start );

    std::cout << "Time: " << duration.count() << " ms" << std::endl;
}

void Problem::PrintResult() const
{
    std::ofstream out( "solution.txt" );
    std::ofstream checkOut( "verification.txt" );
    if ( !out )
    {
        std::cerr << "Cannot open solution.txt for writing\n";
        return;
    }
}

```

```

out << std::scientific << std::setprecision( 12 );
out << "# x u_s u_c\n";
if ( checkOut )
{
    checkOut << std::scientific << std::setprecision( 12 );
    checkOut << "# x us_num us_exact err_us uc_num uc_exact err_uc\n";
}

double maxErrUs = 0.0;
double maxErrUc = 0.0;
const double zLeft = mesh.meshX.front( );
const double zRight = mesh.meshX.back( );
const double len = ( std::abs( zRight - zLeft ) > 0.0 ) ? ( zRight - zLeft ) : 1.0;

for ( int node = 0; node < mesh.n_x; ++node )
{
    const double x = mesh.meshX[node];
    const double usNum = q[2 * node];
    const double ucNum = q[2 * node + 1];
    const double t = ( x - zLeft ) / len;
    const double usRef = 0.2 + 0.6 * t;
    const double ucRef = -0.4 + 0.8 * t;
    const double errUs = std::abs( usNum - usRef );
    const double errUc = std::abs( ucNum - ucRef );

    maxErrUs = std::max( maxErrUs, errUs );
    maxErrUc = std::max( maxErrUc, errUc );

    out << x << " " << usNum << " " << ucNum << "\n";
    if ( checkOut )
    {
        checkOut << x << " " << usNum << " " << usRef << " " << errUs
            << " " << ucNum << " " << ucRef << " " << errUc << "\n";
    }
}

std::cout << "Solution written to solution.txt\n";
if ( checkOut )
{
    std::cout << "Verification written to verification.txt\n";
}

std::cout << "Max |us-us_exact| = " << std::scientific << maxErrUs << "\n";
std::cout << "Max |uc-uc_exact| = " << std::scientific << maxErrUc << "\n";
}

void Problem::Run3DHarmonicTest( int testNumber )
{
    Input3D( testNumber );
    BuildMatrix3D( );
    BuildB3D( );
    Solve3D( );
    PrintResult3D( );
}

void Problem::Input3D( int testNumber )
{
    current3DTestNumber = testNumber;
    mesh3d.InputMeshes( );

    std::ifstream solverFile( "solver.txt" );
    std::ifstream coeffsFile( "coefficients.txt" );
    if ( solverFile )
    {
        std::string mode;
        solverFile >> mode;
        std::transform( mode.begin( ), mode.end( ), mode.begin( ), []( unsigned char c ) {
return static_cast<char>( std::toupper( c ) ); } );
        if ( mode == "MSG" )

```

```

    {
        solveMethod = SolveMethod::MSGDiagonal;
    }
    else
    {
        solveMethod = SolveMethod::LU;
    }
}

if ( coeffsFile )
{
    coeffsFile >> func.lambdaVal >> func.wVal >> func.sigmaVal >> func.xiVal;
}

const int nNodes = static_cast<int>( mesh3d.nodes.size( ) );
n = 2 * nNodes;
matrix = SparseMatrix( n );
b.assign( n, 0.0 );
q3d.assign( n, 0.0 );
usExact3D.assign( nNodes, 0.0 );
ucExact3D.assign( nNodes, 0.0 );
elements3D.resize( mesh3d.hexes.size( ) );
const int variant = current3DTestNumber;

for ( int i = 0; i < nNodes; ++i )
{
    const auto &p = mesh3d.nodes[i];
    usExact3D[i] = func.us3DExact( variant, p.x, p.y, p.z );
    ucExact3D[i] = func.uc3DExact( variant, p.x, p.y, p.z );
}

for ( size_t e = 0; e < mesh3d.hexes.size( ); ++e )
{
    const auto &hex = mesh3d.hexes[e];
    std::array<Element3D::Point3, 8> coords { };
    std::vector<double> localFs( 8, 0.0 );
    std::vector<double> localFc( 8, 0.0 );
    std::vector<double> localLambda( 8, 0.0 );
    std::vector<double> localW( 8, 0.0 );
    std::vector<double> localSigma( 8, 0.0 );
    std::vector<double> localXi( 8, 0.0 );

    for ( int i = 0; i < 8; ++i )
    {
        const auto &p = mesh3d.nodes[hex.v[i]];
        coords[i] = { p.x, p.y, p.z };
        localLambda[i] = func.lambda3D( variant, p.x, p.y, p.z );
        localW[i] = func.w3D( variant, p.x, p.y, p.z );
        localSigma[i] = func.sigma3D( variant, p.x, p.y, p.z );
        localXi[i] = func.xi3D( variant, p.x, p.y, p.z );
        localFs[i] = func.fs3D( variant, p.x, p.y, p.z );
        localFc[i] = func.fc3D( variant, p.x, p.y, p.z );
    }

    elements3D[e] = Element3D( coords, localFs, localFc, localLambda, localW, localSigma,
localXi );
}
}

void Problem::BuildMatrix3D( )
{
    std::vector<std::set<int>> rowProfile( n );
    for ( const auto &hex : mesh3d.hexes )
    {
        std::array<int, 16> map { };
        for ( int i = 0; i < 8; ++i )
        {
            map[2 * i] = 2 * hex.v[i];
            map[2 * i + 1] = 2 * hex.v[i] + 1;
        }
    }
}

```

```

    }

    for ( int i = 0; i < 16; ++i )
    {
        for ( int j = 0; j < 16; ++j )
        {
            if ( map[j] < map[i] )
            {
                rowProfile[map[i]].insert( map[j] );
            }
        }
    }

    matrix.ig.assign( n + 1, 0 );
    for ( int i = 0; i < n; ++i )
    {
        matrix.ig[i + 1] = matrix.ig[i] + static_cast<int>( rowProfile[i].size( ) );
    }

    const int nnz = matrix.ig[n];
    matrix.jg.assign( nnz, 0 );
    matrix.ggl.assign( nnz, 0.0 );
    matrix.ggu.assign( nnz, 0.0 );
    matrix.di.assign( n, 0.0 );

    for ( int i = 0; i < n; ++i )
    {
        int k = matrix.ig[i];
        for ( const int col : rowProfile[i] )
        {
            matrix.jg[k++] = col;
        }
    }

    auto addToGlobal = [this]( int row, int col, double value )
    {
        if ( row == col )
        {
            matrix.di[row] += value;
            return;
        }

        if ( row > col )
        {
            for ( int k = matrix.ig[row]; k < matrix.ig[row + 1]; ++k )
            {
                if ( matrix.jg[k] == col )
                {
                    matrix.ggl[k] += value;
                    return;
                }
            }
        }
        else
        {
            for ( int k = matrix.ig[col]; k < matrix.ig[col + 1]; ++k )
            {
                if ( matrix.jg[k] == row )
                {
                    matrix.ggu[k] += value;
                    return;
                }
            }
        }

        throw std::runtime_error( "Sparse profile mismatch while assembling 3D global
matrix" );
    };

```

```

for ( size_t e = 0; e < mesh3d.hexes.size( ); ++e )
{
    elements3D[e].BuildLocalMatrix( );
    std::array<int, 16> map { };
    for ( int i = 0; i < 8; ++i )
    {
        map[2 * i] = 2 * mesh3d.hexes[e].v[i];
        map[2 * i + 1] = 2 * mesh3d.hexes[e].v[i] + 1;
    }

    for ( int i = 0; i < 16; ++i )
    {
        for ( int j = 0; j < 16; ++j )
        {
            addToGlobal( map[i], map[j], elements3D[e].localMatrix[i][j] );
        }
    }
}

void Problem::BuildB3D( )
{
    std::fill( b.begin( ), b.end( ), 0.0 );
    for ( size_t e = 0; e < mesh3d.hexes.size( ); ++e )
    {
        elements3D[e].BuildLocalB( );
        std::array<int, 16> map { };
        for ( int i = 0; i < 8; ++i )
        {
            map[2 * i] = 2 * mesh3d.hexes[e].v[i];
            map[2 * i + 1] = 2 * mesh3d.hexes[e].v[i] + 1;
        }

        for ( int i = 0; i < 16; ++i )
        {
            b[map[i]] += elements3D[e].localB[i];
        }
    }
}

void Problem::ApplyDirichlet3D( int dof, double value )
{
    ApplyDirichlet( dof, value );
}

void Problem::Solve3D( )
{
    const double xMin = mesh3d.meshX.front( ), xMax = mesh3d.meshX.back( );
    const double yMin = mesh3d.meshY.front( ), yMax = mesh3d.meshY.back( );
    const double zMin = mesh3d.meshZ.front( ), zMax = mesh3d.meshZ.back( );
    const int nNodes = static_cast< int >( mesh3d.nodes.size( ) );
    std::vector<char> isDir( n, 0 );
    std::vector<double> dirVal( n, 0.0 );

    for ( int node = 0; node < nNodes; ++node )
    {
        const auto &p = mesh3d.nodes[node];
        const bool isBoundary =
            std::abs( p.x - xMin ) < 1e-12 || std::abs( p.x - xMax ) < 1e-12 ||
            std::abs( p.y - yMin ) < 1e-12 || std::abs( p.y - yMax ) < 1e-12 ||
            std::abs( p.z - zMin ) < 1e-12 || std::abs( p.z - zMax ) < 1e-12;
        if ( !isBoundary ) continue;

        isDir[2 * node] = 1; dirVal[2 * node] = usExact3D[node];
        isDir[2 * node + 1] = 1; dirVal[2 * node + 1] = ucExact3D[node];
    }

    std::vector<int> oldToNew( n, -1 );
    std::vector<int> newToOld;

```

```

newToOld.reserve( n );
for ( int i = 0; i < n; ++i )
{
    if ( !isDir[i] )
    {
        oldToNew[i] = static_cast< int >( newToOld.size( ) );
        newToOld.push_back( i );
    }
}
const int nFree = static_cast< int >( newToOld.size( ) );
std::vector<double> bFree( nFree, 0.0 );
for ( int k = 0; k < nFree; ++k )
    bFree[k] = b[newToOld[k]];
for ( int row = 0; row < n; ++row )
{
    if ( isDir[row] ) continue;
    const int kRow = oldToNew[row];
    for ( int idx = matrix.ig[row]; idx < matrix.ig[row + 1]; ++idx )
    {
        const int col = matrix.jg[idx];
        if ( isDir[col] )
            bFree[kRow] -= matrix.ggl[idx] * dirVal[col];
    }
}
for ( int col = 0; col < n; ++col )
{
    if ( !isDir[col] ) continue;
    for ( int idx = matrix.ig[col]; idx < matrix.ig[col + 1]; ++idx )
    {
        const int row = matrix.jg[idx]; // row < col
        if ( isDir[row] ) continue;
        bFree[oldToNew[row]] -= matrix.ggu[idx] * dirVal[col];
    }
}
std::vector<int> igR( nFree + 1, 0 );
for ( int k = 0; k < nFree; ++k )
{
    const int row = newToOld[k];
    int cnt = 0;
    for ( int idx = matrix.ig[row]; idx < matrix.ig[row + 1]; ++idx )
    {
        const int col = matrix.jg[idx];
        if ( !isDir[col] )
            ++cnt;
    }
    igR[k + 1] = igR[k] + cnt;
}
const int nnzR = igR[nFree];

SparseMatrix reduced( nFree );
reduced.ig = igR;
reduced.jg.resize( nnzR );
reduced.ggl.resize( nnzR, 0.0 );
reduced.ggu.resize( nnzR, 0.0 );
reduced.di.assign( nFree, 0.0 );

for ( int k = 0; k < nFree; ++k )
{
    const int row = newToOld[k];
    int pos = igR[k];
    for ( int idx = matrix.ig[row]; idx < matrix.ig[row + 1]; ++idx )
    {
        const int col = matrix.jg[idx];
        if ( isDir[col] ) continue;
        reduced.jg[pos] = oldToNew[col];
        reduced.ggl[pos] = matrix.ggl[idx];
        reduced.ggu[pos] = matrix.ggu[idx];
        ++pos;
    }
}

```



```

        reduced.di[k] = matrix.di[row];
    }
    std::vector<double> qFree( nFree, 0.0 );
    auto start = std::chrono::steady_clock::now();

    if ( solveMethod == SolveMethod::MSGDiagonal )
    {
        SLE sle;
        sle.matrix = reduced;
        sle.f = bFree;
        sle.x.assign( nFree, 0.0 );
        sle.maxIter = 1000;
        sle.eps = 1e-15;
        qFree = MSG::SolutionWithDiagonalConditioning( sle );
        std::cout << "Solver: MSG (diagonal preconditioning)\n";
    }
    else
    {
        LU lu( reduced );
        std::vector<double> y( nFree, 0.0 );
        SLEAssistant::LUDirect( lu, bFree, y );
        SLEAssistant::LUReverse( lu, y, qFree );
        std::cout << "Solver: LU\n";
    }

    auto end = std::chrono::steady_clock::now();
    std::cout << "Time: "
        << std::chrono::duration_cast< std::chrono::milliseconds >( end - start ).count()
        << " ms\n";
    q3d.assign( n, 0.0 );
    for ( int k = 0; k < nFree; ++k )
        q3d[newToOld[k]] = qFree[k];
    for ( int i = 0; i < n; ++i )
        if ( isDir[i] )
            q3d[i] = dirVal[i];
}

void Problem::PrintResult3D( )
{
    std::ofstream out( "solution.txt" );
    std::ofstream checkOut( "verification.txt" );
    const double tEval = 1.0;
    out << std::scientific << std::setprecision( 12 );
    out << "# x y z u_s u_c u(t)\n";
    if ( checkOut )
    {
        checkOut << std::scientific << std::setprecision( 12 );
        checkOut << "# x y z us_num us_exact err_us uc_num uc_exact err_uc u_num u_exact
err_u\n";
    }

    double maxErrUs = 0.0;
    double maxErrUc = 0.0;
    double maxErrU = 0.0;
    const int nNodes = static_cast<int>( mesh3d.nodes.size() );
    const int variant = current3DTestNumber;
    const L2Errors3D l2 = ComputeL2Errors3D( tEval );
    for ( int i = 0; i < nNodes; ++i )
    {
        const auto &p = mesh3d.nodes[i];
        const double usNum = q3d[2 * i];
        const double ucNum = q3d[2 * i + 1];
        const double usEx = usExact3D[i];
        const double ucEx = ucExact3D[i];
        const double errUs = std::abs( usNum - usEx );
        const double errUc = std::abs( ucNum - ucEx );
        maxErrUs = std::max( maxErrUs, errUs );
        maxErrUc = std::max( maxErrUc, errUc );
    }
}

```

```

const double w = func.w3D( variant, p.x, p.y, p.z );
const double uNum = usNum * std::sin( w * tEval ) + ucNum * std::cos( w * tEval );
const double uEx = usEx * std::sin( w * tEval ) + ucEx * std::cos( w * tEval );
const double errU = std::abs( uNum - uEx );
maxErrU = std::max( maxErrU, errU );

out << p.x << " " << p.y << " " << p.z << " " << usNum << " " << ucNum << " " << uNum
<< "\n";
if ( checkOut )
{
    checkOut << p.x << " " << p.y << " " << p.z
    << " " << usNum << " " << usEx << " " << errUs
    << " " << ucNum << " " << ucEx << " " << errUc
    << " " << uNum << " " << uEx << " " << errU << "\n";
}
}

std::cout << "Solution written to solution.txt\n";
std::cout << "Verification written to verification.txt\n";
std::cout << "3D harmonic test #" << current3DTestNumber << " (t = " << std::scientific
<< tEval << ")\n";
std::cout << "Max |us-us_exact| = " << std::scientific << maxErrUs << "\n";
std::cout << "Max |uc-uc_exact| = " << std::scientific << maxErrUc << "\n";
std::cout << "Max |u-u_exact| = " << std::scientific << maxErrU << "\n";
std::cout << "|us* - us|L2 = " << std::scientific << l2.us << "\n";
std::cout << "|uc* - uc|L2 = " << std::scientific << l2.uc << "\n";
std::cout << "|u* - u|L2 = " << std::scientific << l2.u << "\n";
}

Problem::L2Errors3D Problem::ComputeL2Errors3D( double t ) const
{
    double l2ErrUs2 = 0.0;
    double l2ErrUc2 = 0.0;
    double l2ErrU2 = 0.0;
    const int variant = current3DTestNumber;
    Basis3D basis3d;
    Quadratures quad;
    const auto quadratures = quad.gaussHex2( );
    auto det3 = [ ]( const std::array<std::array<double, 3>, 3> &m ) -> double
    {
        return
            m[0][0] * ( m[1][1] * m[2][2] - m[1][2] * m[2][1] ) -
            m[0][1] * ( m[1][0] * m[2][2] - m[1][2] * m[2][0] ) +
            m[0][2] * ( m[1][0] * m[2][1] - m[1][1] * m[2][0] );
    };

    for ( size_t e = 0; e < mesh3d.hexes.size( ); ++e )
    {
        const auto &hex = mesh3d.hexes[e];
        for ( const auto &qv : quadratures )
        {
            const std::array<double, 3> rst = { qv[0], qv[1], qv[2] };
            std::array<double, 8> N{ };
            std::array<std::array<double, 3>, 8> gradRef{ };
            for ( int a = 0; a < 8; ++a )
            {
                N[a] = basis3d.Basis( a, rst );
                gradRef[a] = basis3d.GradBasis( a, rst );
            }

            std::array<std::array<double, 3>, 3> J{ };
            double xq = 0.0, yq = 0.0, zq = 0.0;
            double usNumQ = 0.0, ucNumQ = 0.0;
            for ( int a = 0; a < 8; ++a )
            {
                const auto &p = mesh3d.nodes[hex.v[a]];
                J[0][0] += p.x * gradRef[a][0];
                J[0][1] += p.x * gradRef[a][1];
                J[0][2] += p.x * gradRef[a][2];
            }
        }
    }
}

```

```

J[1][0] += p.y * gradRef[a][0];
J[1][1] += p.y * gradRef[a][1];
J[1][2] += p.y * gradRef[a][2];
J[2][0] += p.z * gradRef[a][0];
J[2][1] += p.z * gradRef[a][1];
J[2][2] += p.z * gradRef[a][2];

xq += N[a] * p.x;
yq += N[a] * p.y;
zq += N[a] * p.z;
usNumQ += N[a] * q3d[2 * hex.v[a]];
ucNumQ += N[a] * q3d[2 * hex.v[a] + 1];
}

const double wj = std::abs( det3( J ) ) * qv[3];
const double usExQ = func.us3DExact( variant, xq, yq, zq );
const double ucExQ = func.uc3DExact( variant, xq, yq, zq );
const double dUs = usNumQ - usExQ;
const double dUc = ucNumQ - ucExQ;
l2ErrUs2 += wj * dUs * dUs;
l2ErrUc2 += wj * dUc * dUc;

const double w = func.w3D( variant, xq, yq, zq );
const double uNumQ = usNumQ * std::sin( w * t ) + ucNumQ * std::cos( w * t );
const double uExQ = usExQ * std::sin( w * t ) + ucExQ * std::cos( w * t );
const double dU = uNumQ - uExQ;
l2ErrU2 += wj * dU * dU;
}
}

const double l2ErrUs = std::sqrt( std::max( 0.0, l2ErrUs2 ) );
const double l2ErrUc = std::sqrt( std::max( 0.0, l2ErrUc2 ) );
const double l2ErrU = std::sqrt( std::max( 0.0, l2ErrU2 ) );
return L2Errors3D{ l2ErrUs, l2ErrUc, l2ErrU };
}

```

6.4 Element3D.h

```

#pragma once
#include "Basis1D.h"
#include "Func.h"
#include "Quadratures.h"
#include <vector>
class Element
{
public:
    Element( ) {}

    Element(std::vector<double> _coords, std::vector<double> fs, std::vector<double> fc )
    {
        coords = _coords;
        fsNodes = fs;
        fcNodes = fc;
        localMatrix = std::vector<std::vector<double>>( 4, std::vector<double>( 4, 0.0 ) );
        localB = std::vector<double>( 4, 0.0 );
        lambdaNodes = { func.lambda( coords[0] ), func.lambda( coords[1] ) };
        wNodes = { func.w( coords[0] ), func.w( coords[1] ) };
        sigmaNodes = { func.sigma( coords[0] ), func.sigma( coords[1] ) };
        xiNodes = { func.xi( coords[0] ), func.xi( coords[1] ) };
        beta = func.beta( 0 );
    }

    std::vector<double> lambdaNodes;
    std::vector<double> wNodes;
    std::vector<double> sigmaNodes;
    std::vector<double> xiNodes;

    double beta;
    double thetaS = 0.0;
    double thetaC = 0.0;

```

```

double uBetaS = 0.0;
double uBetaC = 0.0;
void CalcCond( );

double Approx( std::vector<double> val, double x );

std::vector<std::vector<double>> localMatrix;
std::vector<double> fsNodes;
std::vector<double> fcNodes;
std::vector<double> localB;
std::vector<double> coords;

void BuildLocalMatrix( );
void BuildLocalB( );
private:
    Func func;
    Quadratures quad;
    Basis1D basis;
};

```

6.5 Element3D.cpp

```

#include "Element3D.h"
#include <cmath>
#include <stdexcept>

double Element3D::Det3( const std::array<std::array<double, 3>, 3> &m )
{
    return
        m[0][0] * ( m[1][1] * m[2][2] - m[1][2] * m[2][1] ) -
        m[0][1] * ( m[1][0] * m[2][2] - m[1][2] * m[2][0] ) +
        m[0][2] * ( m[1][0] * m[2][1] - m[1][1] * m[2][0] );
}

std::array<std::array<double, 3>, 3> Element3D::Inverse3( const
std::array<std::array<double, 3>, 3> &m )
{
    const double det = Det3( m );
    if ( std::abs( det ) < 1e-20 )
    {
        throw std::runtime_error( "Degenerate hexahedron: Jacobian determinant is near zero"
    );
    }

    std::array<std::array<double, 3>, 3> inv {};
    inv[0][0] = ( m[1][1] * m[2][2] - m[1][2] * m[2][1] ) / det;
    inv[0][1] = ( m[0][2] * m[2][1] - m[0][1] * m[2][2] ) / det;
    inv[0][2] = ( m[0][1] * m[1][2] - m[0][2] * m[1][1] ) / det;

    inv[1][0] = ( m[1][2] * m[2][0] - m[1][0] * m[2][2] ) / det;
    inv[1][1] = ( m[0][0] * m[2][2] - m[0][2] * m[2][0] ) / det;
    inv[1][2] = ( m[0][2] * m[1][0] - m[0][0] * m[1][2] ) / det;

    inv[2][0] = ( m[1][0] * m[2][1] - m[1][1] * m[2][0] ) / det;
    inv[2][1] = ( m[0][1] * m[2][0] - m[0][0] * m[2][1] ) / det;
    inv[2][2] = ( m[0][0] * m[1][1] - m[0][1] * m[1][0] ) / det;
    return inv;
}

double Element3D::Approx( const std::vector<double> &val, const std::array<double, 3> &rst
) const
{
    if ( val.size( ) < 8 )
    {
        return 0.0;
    }
    double res = 0.0;
    for ( int i = 0; i < 8; ++i )

```

```

{
    res += val[i] * basis.Basis( i, rst );
}
return res;
}

void Element3D::BuildLocalMatrix( )
{
    localMatrix.assign( 16, std::vector<double>( 16, 0.0 ) );

    const std::vector<std::vector<double>> quadratures = quad.gaussHex2( );
    for ( const auto &qv : quadratures )
    {
        const std::array<double, 3> q = { qv[0], qv[1], qv[2] };
        std::array<std::array<double, 3>, 3> J {};
        std::array<std::array<double, 3>, 8> gradRef {};
        for ( int aNode = 0; aNode < 8; ++aNode )
        {
            gradRef[aNode] = basis.GradBasis( aNode, q );
            J[0][0] += coords[aNode][0] * gradRef[aNode][0];
            J[0][1] += coords[aNode][0] * gradRef[aNode][1];
            J[0][2] += coords[aNode][0] * gradRef[aNode][2];
            J[1][0] += coords[aNode][1] * gradRef[aNode][0];
            J[1][1] += coords[aNode][1] * gradRef[aNode][1];
            J[1][2] += coords[aNode][1] * gradRef[aNode][2];
            J[2][0] += coords[aNode][2] * gradRef[aNode][0];
            J[2][1] += coords[aNode][2] * gradRef[aNode][1];
            J[2][2] += coords[aNode][2] * gradRef[aNode][2];
        }

        const double detJ = Det3( J );
        const auto invJ = Inverse3( J );
        const double wj = std::abs( detJ ) * qv[3];
        const double lambda = Approx( lambdaNodes, q );
        const double w = Approx( wNodes, q );
        const double sigma = Approx( sigmaNodes, q );
        const double xi = Approx( xiNodes, q );

        std::array<std::array<double, 3>, 8> gradPhys {};
        for ( int i = 0; i < 8; ++i )
        {
            for ( int c = 0; c < 3; ++c )
            {
                gradPhys[i][c] =
                    invJ[0][c] * gradRef[i][0] +
                    invJ[1][c] * gradRef[i][1] +
                    invJ[2][c] * gradRef[i][2];
            }
        }

        for ( int i = 0; i < 8; ++i )
        {
            for ( int j = 0; j < 8; ++j )
            {
                const double dot =
                    gradPhys[i][0] * gradPhys[j][0] +
                    gradPhys[i][1] * gradPhys[j][1] +
                    gradPhys[i][2] * gradPhys[j][2];
                const double psiI = basis.Basis( i, q );
                const double psiJ = basis.Basis( j, q );
                const double a = wj * ( lambda * dot - w * w * xi * psiI * psiJ );
                const double b = wj * w * sigma * psiI * psiJ;

                localMatrix[2 * i][2 * j] += a;
                localMatrix[2 * i][2 * j + 1] += -b;
                localMatrix[2 * i + 1][2 * j] += b;
                localMatrix[2 * i + 1][2 * j + 1] += a;
            }
        }
    }
}

```

```

    }
}

void Element3D::BuildLocalB( )
{
    localB.assign( 16, 0.0 );

    const std::vector<std::vector<double>> quadratures = quad.gaussHex2( );
    for ( const auto &qv : quadratures )
    {
        const std::array<double, 3> q = { qv[0], qv[1], qv[2] };
        std::array<std::array<double, 3>, 3> J {};
        for ( int aNode = 0; aNode < 8; ++aNode )
        {
            const auto g = basis.GradBasis( aNode, q );
            J[0][0] += coords[aNode][0] * g[0];
            J[0][1] += coords[aNode][0] * g[1];
            J[0][2] += coords[aNode][0] * g[2];
            J[1][0] += coords[aNode][1] * g[0];
            J[1][1] += coords[aNode][1] * g[1];
            J[1][2] += coords[aNode][1] * g[2];
            J[2][0] += coords[aNode][2] * g[0];
            J[2][1] += coords[aNode][2] * g[1];
            J[2][2] += coords[aNode][2] * g[2];
        }

        const double detJ = Det3( J );
        const double wj = std::abs( detJ ) * qv[3];
        const double fs = Approx( fsNodes, q );
        const double fc = Approx( fcNodes, q );
        for ( int i = 0; i < 8; ++i )
        {
            const double Ni = basis.Basis( i, q );
            localB[2 * i] += wj * fs * Ni;
            localB[2 * i + 1] += wj * fc * Ni;
        }
    }
}

```

6.6 Basis3D.h

```

#pragma once
#include <array>

class Basis3D
{
public:
    double Basis1( const std::array<double, 3> &p ) const;
    double Basis2( const std::array<double, 3> &p ) const;
    double Basis3( const std::array<double, 3> &p ) const;
    double Basis4( const std::array<double, 3> &p ) const;
    double Basis5( const std::array<double, 3> &p ) const;
    double Basis6( const std::array<double, 3> &p ) const;
    double Basis7( const std::array<double, 3> &p ) const;
    double Basis8( const std::array<double, 3> &p ) const;

    std::array<double, 3> GradBasis1( const std::array<double, 3> &p ) const;
    std::array<double, 3> GradBasis2( const std::array<double, 3> &p ) const;
    std::array<double, 3> GradBasis3( const std::array<double, 3> &p ) const;
    std::array<double, 3> GradBasis4( const std::array<double, 3> &p ) const;
    std::array<double, 3> GradBasis5( const std::array<double, 3> &p ) const;
    std::array<double, 3> GradBasis6( const std::array<double, 3> &p ) const;
    std::array<double, 3> GradBasis7( const std::array<double, 3> &p ) const;
    std::array<double, 3> GradBasis8( const std::array<double, 3> &p ) const;

    double Basis( int i, const std::array<double, 3> &p ) const;
    std::array<double, 3> GradBasis( int i, const std::array<double, 3> &p ) const;
};

```

6.7 Basis3D.cpp

```

#include "Basis3D.h"

double Basis3D::Basis1( const std::array<double, 3> &p ) const
{
    const double r = p[0], s = p[1], t = p[2];
    return ( 1.0 - r ) * ( 1.0 - s ) * ( 1.0 - t );
}

double Basis3D::Basis2( const std::array<double, 3> &p ) const
{
    const double r = p[0], s = p[1], t = p[2];
    return r * ( 1.0 - s ) * ( 1.0 - t );
}

double Basis3D::Basis3( const std::array<double, 3> &p ) const
{
    const double r = p[0], s = p[1], t = p[2];
    return r * s * ( 1.0 - t );
}

double Basis3D::Basis4( const std::array<double, 3> &p ) const
{
    const double r = p[0], s = p[1], t = p[2];
    return ( 1.0 - r ) * s * ( 1.0 - t );
}

double Basis3D::Basis5( const std::array<double, 3> &p ) const
{
    const double r = p[0], s = p[1], t = p[2];
    return ( 1.0 - r ) * ( 1.0 - s ) * t;
}

double Basis3D::Basis6( const std::array<double, 3> &p ) const
{
    const double r = p[0], s = p[1], t = p[2];
    return r * ( 1.0 - s ) * t;
}

double Basis3D::Basis7( const std::array<double, 3> &p ) const
{
    const double r = p[0], s = p[1], t = p[2];
    return r * s * t;
}

double Basis3D::Basis8( const std::array<double, 3> &p ) const
{
    const double r = p[0], s = p[1], t = p[2];
    return ( 1.0 - r ) * s * t;
}

std::array<double, 3> Basis3D::GradBasis1( const std::array<double, 3> &p ) const
{
    const double r = p[0], s = p[1], t = p[2];
    return { -( 1.0 - s ) * ( 1.0 - t ), -( 1.0 - r ) * ( 1.0 - t ), -( 1.0 - r ) * ( 1.0 - s ) };
}

std::array<double, 3> Basis3D::GradBasis2( const std::array<double, 3> &p ) const
{
    const double r = p[0], s = p[1], t = p[2];
    return { ( 1.0 - s ) * ( 1.0 - t ), -r * ( 1.0 - t ), -r * ( 1.0 - s ) };
}

std::array<double, 3> Basis3D::GradBasis3( const std::array<double, 3> &p ) const
{
    const double r = p[0], s = p[1], t = p[2];
    return { s * ( 1.0 - t ), r * ( 1.0 - t ), -r * s };
}

```

```

std::array<double, 3> Basis3D::GradBasis4( const std::array<double, 3> &p ) const
{
    const double r = p[0], s = p[1], t = p[2];
    return { -s * ( 1.0 - t ), ( 1.0 - r ) * ( 1.0 - t ), -( 1.0 - r ) * s };
}

std::array<double, 3> Basis3D::GradBasis5( const std::array<double, 3> &p ) const
{
    const double r = p[0], s = p[1], t = p[2];
    return { -( 1.0 - s ) * t, -( 1.0 - r ) * t, ( 1.0 - r ) * ( 1.0 - s ) };
}

std::array<double, 3> Basis3D::GradBasis6( const std::array<double, 3> &p ) const
{
    const double r = p[0], s = p[1], t = p[2];
    return { ( 1.0 - s ) * t, -r * t, r * ( 1.0 - s ) };
}

std::array<double, 3> Basis3D::GradBasis7( const std::array<double, 3> &p ) const
{
    const double r = p[0], s = p[1], t = p[2];
    return { s * t, r * t, r * s };
}

std::array<double, 3> Basis3D::GradBasis8( const std::array<double, 3> &p ) const
{
    const double r = p[0], s = p[1], t = p[2];
    return { -s * t, ( 1.0 - r ) * t, ( 1.0 - r ) * s };
}

double Basis3D::Basis( int i, const std::array<double, 3> &p ) const
{
    switch ( i )
    {
        case 0: return Basis1( p );
        case 1: return Basis2( p );
        case 2: return Basis3( p );
        case 3: return Basis4( p );
        case 4: return Basis5( p );
        case 5: return Basis6( p );
        case 6: return Basis7( p );
        case 7: return Basis8( p );
        default: return 0.0;
    }
}

std::array<double, 3> Basis3D::GradBasis( int i, const std::array<double, 3> &p ) const
{
    switch ( i )
    {
        case 0: return GradBasis1( p );
        case 1: return GradBasis2( p );
        case 2: return GradBasis3( p );
        case 3: return GradBasis4( p );
        case 4: return GradBasis5( p );
        case 5: return GradBasis6( p );
        case 6: return GradBasis7( p );
        case 7: return GradBasis8( p );
        default: return { 0.0, 0.0, 0.0 };
    }
}

6.8 Quadratures.h
#pragma once
#include <vector>
class Quadratures
{
public:
    std::vector<std::vector<double>> gauss3();

```



```

std::vector<std::vector<double>> gaussHex2();
};

6.9 Quadratures.cpp
#include "Quadratures.h"
#include <cmath>

std::vector<std::vector<double>> Quadratures::gauss3( )
{
    std::vector<std::vector<double>> res( 3 );
    double t1 = -std::sqrt( 15.0 ) / 5.0;
    double t2 = 0.0;
    double t3 = +std::sqrt( 15.0 ) / 5.0;
    double w1 = 5.0 / 9.0;
    double w2 = 8.0 / 9.0;
    double w3 = 5.0 / 9.0;

    res[0] = std::vector<double>{ ( t1 + 1 ) / 2, 0.5 * w1 };
    res[1] = std::vector<double>{ ( t2 + 1 ) / 2, 0.5 * w2 };
    res[2] = std::vector<double>{ ( t3 + 1 ) / 2, 0.5 * w3 };
    return res;
}

std::vector<std::vector<double>> Quadratures::gaussHex2( )
{
    const double a = 1.0 / std::sqrt( 3.0 );
    const double p0 = 0.5 * ( 1.0 - a );
    const double p1 = 0.5 * ( 1.0 + a );
    const double w = 0.125; // (1/2)^3 для перехода [-1,1]^3 -> [0,1]^3

    std::vector<std::vector<double>> q;
    q.reserve( 8 );
    for ( int ir = 0; ir < 2; ++ir )
    {
        for ( int is = 0; is < 2; ++is )
        {
            for ( int it = 0; it < 2; ++it )
            {
                q.push_back( { ir ? p1 : p0, is ? p1 : p0, it ? p1 : p0, w } );
            }
        }
    }
    return q;
}

6.10 Mesh3D.h
#pragma once
#include "Mesh.h"
#include <array>
#include <vector>

class Mesh3D : public Mesh
{
public:
    struct Node3D
    {
        double x = 0.0;
        double y = 0.0;
        double z = 0.0;
    };

    struct Hex8
    {
        std::array<int, 8> v {};
    };

    std::vector<double> meshY;
    std::vector<double> meshZ;
};

```

```

int n_y = 0;
int n_z = 0;
bool uniformY = true;
bool uniformZ = true;

std::vector<Node3D> nodes;
std::vector<Hex8> hexes;

void InputMeshes( );
void BuildStructuredHexMesh( );
};

        6.11 Mesh3D.cpp
#include "Mesh3D.h"
#include <fstream>
#include <iostream>

void Mesh3D::InputMeshes( )
{
    std::ifstream f( "mesh3d.txt" );
    if ( !f )
    {
        std::cerr << "File open error: mesh3d.txt\n";
        return;
    }

    double ax, bx, qx;
    double ay, by, qy;
    double az, bz, qz;

    f >> uniformX >> n_x >> ax >> bx;
    f >> uniformY >> n_y >> ay >> by;
    f >> uniformZ >> n_z >> az >> bz;

    meshX.resize( n_x );
    meshY.resize( n_y );
    meshZ.resize( n_z );

    if ( uniformX )
    {
        CalculateUniformMesh( meshX, ax, bx, n_x );
    }
    else
    {
        f >> qx;
        CalculateNonUniformMesh( meshX, ax, bx, n_x, qx );
    }

    if ( uniformY )
    {
        CalculateUniformMesh( meshY, ay, by, n_y );
    }
    else
    {
        f >> qy;
        CalculateNonUniformMesh( meshY, ay, by, n_y, qy );
    }

    if ( uniformZ )
    {
        CalculateUniformMesh( meshZ, az, bz, n_z );
    }
    else
    {
        f >> qz;
        CalculateNonUniformMesh( meshZ, az, bz, n_z, qz );
    }

    BuildStructuredHexMesh( );
}

```

```

}

void Mesh3D::BuildStructuredHexMesh( )
{
    nodes.clear( );
    nodes.reserve( n_x * n_y * n_z );
    for ( int k = 0; k < n_z; ++k )
    {
        for ( int j = 0; j < n_y; ++j )
        {
            for ( int i = 0; i < n_x; ++i )
            {
                nodes.push_back( { meshX[i], meshY[j], meshZ[k] } );
            }
        }
    }

    auto idx = [this]( int i, int j, int k ) -> int
    {
        return i + j * n_x + k * n_x * n_y;
    };

    hexes.clear( );
    for ( int k = 0; k < n_z - 1; ++k )
    {
        for ( int j = 0; j < n_y - 1; ++j )
        {
            for ( int i = 0; i < n_x - 1; ++i )
            {
                const int v000 = idx( i, j, k );
                const int v100 = idx( i + 1, j, k );
                const int v110 = idx( i + 1, j + 1, k );
                const int v010 = idx( i, j + 1, k );
                const int v001 = idx( i, j, k + 1 );
                const int v101 = idx( i + 1, j, k + 1 );
                const int v111 = idx( i + 1, j + 1, k + 1 );
                const int v011 = idx( i, j + 1, k + 1 );
                hexes.push_back( { { v000, v100, v110, v010, v001, v101, v111, v011 } } );
            }
        }
    }
}

```

6.12 SparseMatrix.h

```
#pragma once
#include <vector>
class SparseMatrix
{
public:
    SparseMatrix( ) {}
    SparseMatrix( int _n )
    {
        n = _n;
        di = std::vector( n, 0.0 );
        ig = std::vector<int>( n + 1, 0 );
        jg.clear( );
        ggl.clear( );
        ggu.clear( );
    }

    int n;

    std::vector<double> di;
    std::vector<int> ig;
    std::vector<int> jg;
    std::vector<double> ggl;
    std::vector<double> ggu;
};
```

6.13 SLE.h

```
#pragma once
#include "SparseMatrix.h"
#include <fstream>

class SLE
{
public:
    int                maxIter = 0;
    double             eps = 0;
    SparseMatrix       matrix;
    std::vector<double> f;
    std::vector<double> x;

    void Input( );

private:
    void InputProperty( );
    void InputSLE( );
};
```

6.14 SLE.cpp

```
#include "SLE.h"
#include <stdexcept>

void SLE::Input( )
{
    InputProperty( );
    InputSLE( );
}

void SLE::InputProperty( )
{
    std::fstream file( "C:\\NumMeth\\data\\property.txt", std::ios::in );
    if ( !file.is_open( ) ) throw std::runtime_error( "The file \"property.txt\" cannot be opened!" );

    file >> matrix.n >> maxIter >> eps;
```

```

    file.close( );
}

void SLE::InputSLE( )
{
    std::ifstream file( "C:\\NumMeth\\data\\ig.txt" );
    if ( !file.is_open( ) ) throw std::runtime_error( "The file \"ig.txt\" cannot be
opened!" );

    matrix.ig.resize( matrix.n + 1 );
    for ( unsigned int i = 0; i <= matrix.n; ++i )
        file >> matrix.ig[i];

    file.close( );
    //-----
    file.open( "C:\\NumMeth\\data\\jg.txt" );
    if ( !file.is_open( ) ) throw std::runtime_error( "The file \"jg.txt\" cannot be
opened!" );

    size_t buf = matrix.ig[matrix.n];
    matrix.jg.resize( buf );
    for ( size_t i = 0; i < buf; ++i )
        file >> matrix.jg[i];

    file.close( );

    if ( matrix.ig[0] )
    {
        for ( int i = 0; i < matrix.ig.size( ); ++i )
        {
            matrix.ig[i]--;
        }
        for ( int j = 0; j < matrix.jg.size( ); ++j )
        {
            matrix.jg[j]--;
        }
    }

    //-----
    matrix.ggl.resize( buf );
    file.open( "C:\\NumMeth\\data\\gg.txt" );
    if ( !file.is_open( ) ) throw std::runtime_error( "The file \"gg.txt\" cannot be opened!"
);
    for ( size_t i = 0; i < buf; ++i )
        file >> matrix.ggl[i];
    file.close( );
    //-----
    file.open( "C:\\NumMeth\\data\\di.txt" );
    if ( !file.is_open( ) ) throw std::runtime_error( "The file \"di.txt\" cannot be
opened!" );

    matrix.di.resize( matrix.n );
    for ( size_t i = 0; i < matrix.n; ++i )
        file >> matrix.di[i];

    file.close( );
    //-----
    file.open( "C:\\NumMeth\\data\\f.txt" );
    if ( !file.is_open( ) ) throw std::runtime_error( "The file \"f.txt\" cannot be opened!"
);

    f.resize( matrix.n );
    for ( size_t i = 0; i < matrix.n; ++i )
        file >> f[i];

    file.close( );
}

```

6.15 SLEAssistant.h

```
#pragma once
#include "SLE.h"
#include "LU.h"
#include "LLt.h"
#include "ILU.h"

class SLEAssistant
{
public:
    static double DotProduct(
        const std::vector<double> &v1,
        const std::vector<double> &v2
    );
    static void MatrixVectorProduct(
        const SparseMatrix &m,
        const std::vector<double> &v,
        std::vector<double> &res
    );
    static void LUDirect(
        const LU &lu,
        const std::vector<double> &v,
        std::vector<double> &res
    );
    static void LUReverse(
        const LU &lu,
        const std::vector<double> &v,
        std::vector<double> &res
    );
    static void DiagonalPreconditionMSG(
        SparseMatrix &M,
        std::vector<double> &v,
        std::vector<double> &res
    );
    static void DiagonalPreconditionLOS(
        SparseMatrix &M,
        std::vector<double> &v,
        std::vector<double> &res
    );
    static void LLtDirect(
        const LLt &llt,
        const std::vector<double> &v,
        std::vector<double> &res
    );
    static void LLtReverse(
        const LLt &llt,
        const std::vector<double> &v,
        std::vector<double> &res
    );
    static void ILUDirect(
        const ILU &ilu,
        const std::vector<double> &v,
        std::vector<double> &res
    );
    static void ILUReverse(
        const ILU &ilu,
        const std::vector<double> &v,
        std::vector<double> &res
    );
};
```

6.16 SLEAssistant.cpp

```
#include "SLEAssistant.h"
#include <algorithm>
#include <cmath>

double SLEAssistant::DotProduct( const std::vector<double> &v1, const std::vector<double>
&v2 )
{
```

```

double sum = 0;
for ( int i = 0; i < v1.size( ); ++i )
    sum += v1[i] * v2[i];
return sum;
}

void SLEAssistant::MatrixVectorProduct( const SparseMatrix &m, const std::vector<double>
&v, std::vector<double> &res )
{
    std::fill( res.begin( ), res.end( ), 0.0 );
    for ( int i = 0; i < m.n; ++i )
    {
        res[i] = m.di[i] * v[i];
        for ( int k = m.ig[i]; k < m.ig[i + 1]; k++ )
        {
            res[i] += m.ggl[k] * v[m.jg[k]];
            res[m.jg[k]] += m.ggu[k] * v[i];
        }
    }
}

void SLEAssistant::LUDirect( const LU &lu, const std::vector<double> &v,
std::vector<double> &res )
{
    double sum = 0;
    res = v;
    for ( int i = 0; i < res.size( ); ++i )
    {
        sum = 0;
        for ( int k = lu.ig[i]; k < lu.ig[i + 1]; ++k )
            sum += lu.ggl[k] * res[lu.jg[k]];
        res[i] -= sum;
    }
}

void SLEAssistant::LUReverse( const LU &lu, const std::vector<double> &v,
std::vector<double> &res )
{
    std::vector<double> z = v;
    for ( int i = z.size( ) - 1; i >= 0; --i )
    {
        res[i] = z[i] / lu.di[i];
        for ( int k = lu.ig[i]; k < lu.ig[i + 1]; ++k )
            z[lu.jg[k]] -= res[i] * lu.ggu[k];
    }
}

void SLEAssistant::DiagonalPreconditionMSG( SparseMatrix &M, std::vector<double> &v,
std::vector<double> &res )
{
    for ( int i = 0; i < M.n; ++i )
        res[i] = v[i] / M.di[i];
}

void SLEAssistant::DiagonalPreconditionLOS( SparseMatrix &M, std::vector<double> &v,
std::vector<double> &res )
{
    for ( int i = 0; i < M.n; ++i )
        res[i] = v[i] / sqrt( M.di[i] );
}

void SLEAssistant::LLtDirect( const LLt &llt, const std::vector<double> &v,
std::vector<double> &res )
{
    res = v;
    for ( int i = 0; i < res.size( ); ++i )
    {
        for ( int k = llt.ig[i]; k < llt.ig[i + 1]; ++k )
            res[i] -= llt.ggl[k] * res[llt.jg[k]];
    }
}

```

```

        res[i] /= llt.di[i];
    }
}

void SLEAssistant::LLtReverse( const LLt &llt, const std::vector<double> &v,
std::vector<double> &res )
{
    res = v;
    for ( int i = res.size() - 1; i >= 0; --i )
    {
        res[i] /= llt.di[i];
        for ( int k = llt.ig[i]; k < llt.ig[i + 1]; ++k )
            res[llt.jg[k]] -= llt.ggl[k] * res[i];
    }
}

void SLEAssistant::ILUDirect( const ILU &ilu,
const std::vector<double> &v,
std::vector<double> &res )
{
    res = v;
    for ( int i = 0; i < ( int )res.size(); ++i )
    {
        double sum = 0.0;
        for ( int k = ilu.ig[i]; k < ilu.ig[i + 1]; ++k )
            sum += ilu.ggl[k] * res[ilu.jg[k]];
        res[i] -= sum;
        // диагональ L = 1, делить не нужно
    }
}

void SLEAssistant::ILUReverse( const ILU &ilu,
const std::vector<double> &v,
std::vector<double> &res )
{
    std::vector<double> z = v;
    for ( int i = ( int )z.size() - 1; i >= 0; --i )
    {
        res[i] = z[i] / ilu.di[i]; // U(i,i)
        for ( int k = ilu.ig[i]; k < ilu.ig[i + 1]; ++k )
            z[ilu.jg[k]] -= res[i] * ilu.ggu[k]; // обратная замена по U
    }
}

```

6.17 LU.h

```

#pragma once

#include "SparseMatrix.h"

class LU : public SparseMatrix
{
public:
    std::vector<double> ggu;

    LU( const SparseMatrix &matrix );
};

```

6.18 LU.cpp

```

#include "LU.h"
#include <cmath>
#include <stdexcept>

LU::LU( const SparseMatrix &matrix )
{
    n = matrix.n;
    di = matrix.di;
    ig = matrix.ig;
    jg = matrix.jg;
    ggl = matrix.ggl;
}

```



```

ggu = matrix.ggu;

for ( int i = 0; i < n; ++i )
{
    for ( int idx = ig[i]; idx < ig[i + 1]; ++idx )
    {
        const int j = jg[idx];

        // Вычисление L(i,j): (A(i,j) - sum_{m<j} L(i,m) * U(m,j)) / D(j)
        double sumL = 0.0;
        int p = ig[i];
        int q = ig[j];
        while ( p < idx && q < ig[j + 1] )
        {
            const int colP = jg[p];
            const int colQ = jg[q];
            if ( colP == colQ )
            {
                sumL += ggl[p] * ggu[q];
                ++p;
                ++q;
            }
            else if ( colP < colQ )
            {
                ++p;
            }
            else
            {
                ++q;
            }
        }
        ggl[idx] = ( ggl[idx] - sumL ) / di[j];

        // Вычисление U(j,i): A(j,i) - sum_{m<j} L(j,m) * U(m,i)
        double sumU = 0.0;
        p = ig[i];
        q = ig[j];
        while ( p < idx && q < ig[j + 1] )
        {
            const int colP = jg[p];
            const int colQ = jg[q];
            if ( colP == colQ )
            {
                sumU += ggu[p] * ggl[q];
                ++p;
                ++q;
            }
            else if ( colP < colQ )
            {
                ++p;
            }
            else
            {
                ++q;
            }
        }
        ggu[idx] = ggu[idx] - sumU;
    }

    double sumD = 0.0;
    for ( int idx = ig[i]; idx < ig[i + 1]; ++idx )
    {
        sumD += ggl[idx] * ggu[idx];
    }
    di[i] -= sumD;
    if ( std::abs( di[i] ) < 1e-20 )
    {
        throw std::runtime_error( "LU factorization failed: near-zero diagonal pivot" );
    }
}

```

```

}
}

6.19 MSG.h
#pragma once
#include "SLE.h"
#include "SLEAssistant.h"
#include <iostream>

class MSG
{
public:
    static std::vector<double> SolutionWithoutConditioning( SLE sle );
    static std::vector<double> SolutionWithDiagonalConditioning( SLE sle );
    static std::vector<double> SolutionWithLUConditioning( SLE sle );
    static std::vector<double> SolutionWithLLtConditioning( SLE sle );
    static std::vector<double> SolutionWithILUConditioning( SLE sle );
};

```

```

6.20 MSG.cpp
#include "MSG.h"
#include <cmath>

namespace
{
    std::vector<double> SolveBiCGStab( SLE sle, bool diagonalPreconditioner )
    {
        const int n = sle.matrix.n;
        const double tiny = 1e-30;

        std::vector<double> x = sle.x;
        std::vector<double> ax( n, 0.0 );
        std::vector<double> r( n, 0.0 );
        std::vector<double> rHat( n, 0.0 );
        std::vector<double> p( n, 0.0 );
        std::vector<double> pHat( n, 0.0 );
        std::vector<double> v( n, 0.0 );
        std::vector<double> s( n, 0.0 );
        std::vector<double> sHat( n, 0.0 );
        std::vector<double> t( n, 0.0 );

        SLEAssistant::MatrixVectorProduct( sle.matrix, x, ax );
        for ( int i = 0; i < n; ++i )
        {
            r[i] = sle.f[i] - ax[i];
        }
        rHat = r;

        const double fNorm2 = SLEAssistant::DotProduct( sle.f, sle.f );
        if ( fNorm2 < tiny )
        {
            std::cout << "Residual: 0 k: 0\n";
            return x;
        }

        const double stop2 = ( sle.eps * sle.eps ) * fNorm2;
        double residual2 = SLEAssistant::DotProduct( r, r );
        if ( residual2 <= stop2 )
        {
            std::cout << "Residual: " << std::sqrt( residual2 / fNorm2 ) << " k: 0\n";
            return x;
        }

        double rhoOld = 1.0;
        double alpha = 1.0;
        double omega = 1.0;
        int k = 0;

        for ( ; k < sle.maxIter; ++k )

```

```

{
    const double rhoNew = SLEAssistant::DotProduct( rHat, r );
    if ( std::abs( rhoNew ) < tiny )
    {
        break;
    }

    if ( k == 0 )
    {
        p = r;
    }
    else
    {
        const double beta = ( rhoNew / rhoOld ) * ( alpha / omega );
        for ( int i = 0; i < n; ++i )
        {
            p[i] = r[i] + beta * ( p[i] - omega * v[i] );
        }
    }

    if ( diagonalPreconditioner )
    {
        SLEAssistant::DiagonalPreconditionMSG( sle.matrix, p, pHat );
    }
    else
    {
        pHat = p;
    }

    SLEAssistant::MatrixVectorProduct( sle.matrix, pHat, v );
    const double denomAlpha = SLEAssistant::DotProduct( rHat, v );
    if ( std::abs( denomAlpha ) < tiny )
    {
        break;
    }
    alpha = rhoNew / denomAlpha;

    for ( int i = 0; i < n; ++i )
    {
        s[i] = r[i] - alpha * v[i];
    }

    const double sNorm2 = SLEAssistant::DotProduct( s, s );
    if ( sNorm2 <= stop2 )
    {
        for ( int i = 0; i < n; ++i )
        {
            x[i] += alpha * pHat[i];
        }
        residual2 = sNorm2;
        ++k;
        break;
    }

    if ( diagonalPreconditioner )
    {
        SLEAssistant::DiagonalPreconditionMSG( sle.matrix, s, sHat );
    }
    else
    {
        sHat = s;
    }

    SLEAssistant::MatrixVectorProduct( sle.matrix, sHat, t );
    const double tt = SLEAssistant::DotProduct( t, t );
    if ( tt < tiny )
    {
        break;
    }
}

```

```

        omega = SLEAssistant::DotProduct( t, s ) / tt;
        if ( std::abs( omega ) < tiny )
        {
            break;
        }

        for ( int i = 0; i < n; ++i )
        {
            x[i] += alpha * pHat[i] + omega * sHat[i];
            r[i] = s[i] - omega * t[i];
        }

        residual2 = SLEAssistant::DotProduct( r, r );
        if ( residual2 <= stop2 )
        {
            ++k;
            break;
        }

        rhoOld = rhoNew;
    }

    std::cout << "Residual: " << std::sqrt( residual2 / fNorm2 ) << " k: " << k << "\n";
    return x;
}

std::vector<double> MSG::SolutionWithoutConditioning( SLE sle )
{
    return SolveBiCGStab( sle, false );
}

std::vector<double> MSG::SolutionWithDiagonalConditioning( SLE sle )
{
    return SolveBiCGStab( sle, true );
}

std::vector<double> MSG::SolutionWithLUConditioning( SLE sle )
{
    LU    lu( sle.matrix );
    int    k = 0;
    double residual = 1;
    double a = 0;
    double b = 0;
    double fNorm = sqrt( SLEAssistant::DotProduct( sle.f, sle.f ) );
    double condition = ( sle.eps * fNorm ) * ( sle.eps * fNorm );

    std::vector<double> ax( sle.matrix.n, 0 );
    std::vector<double> z( sle.matrix.n, 0 );
    std::vector<double> az( sle.matrix.n, 0 );
    std::vector<double> rLU( sle.matrix.n, 0 );
    std::vector<double> r = sle.f;

    SLEAssistant::LUDirect( lu, r, z );
    SLEAssistant::LUReverse( lu, z, z );

    for ( ; k < sle.maxIter and residual > condition; ++k )
    {
        SLEAssistant::MatrixVectorProduct( sle.matrix, z, az );
        SLEAssistant::LUDirect( lu, r, rLU );
        SLEAssistant::LUReverse( lu, rLU, rLU );
        b = SLEAssistant::DotProduct( rLU, r );
        a = b / SLEAssistant::DotProduct( az, z );

        for ( int i = 0; i < sle.matrix.n; ++i )
        {
            sle.x[i] = sle.x[i] + a * z[i];
            r[i] = r[i] - a * az[i];
        }
    }
}

```

```

    SLEAssistant::LUDirect( lu, r, rLU );
    SLEAssistant::LUReverse( lu, rLU, rLU );
    b = SLEAssistant::DotProduct( rLU, r ) / b;

    for ( int i = 0; i < sle.matrix.n; ++i )
        z[i] = rLU[i] + b * z[i];

    residual = SLEAssistant::DotProduct( r, r );
}

std::cout << "Residual: " << sqrt( residual ) / fNorm << " k: " << k << "\n";
return sle.x;
}

std::vector<double> MSG::SolutionWithLLtConditioning( SLE sle )
{
    LLt    llt( sle.matrix );
    int    k = 0;
    double residual = 1;
    double a = 0;
    double b = 0;
    double fNorm = sqrt( SLEAssistant::DotProduct( sle.f, sle.f ) );
    double condition = ( sle.eps * fNorm ) * ( sle.eps * fNorm );

    std::vector<double> ax( sle.matrix.n, 0 );
    std::vector<double> z( sle.matrix.n, 0 );
    std::vector<double> az( sle.matrix.n, 0 );
    std::vector<double> rLLt( sle.matrix.n, 0 );
    std::vector<double> r = sle.f;

    SLEAssistant::LLtDirect( llt, r, z );
    SLEAssistant::LLtReverse( llt, z, z );

    bool nan = false;

    for ( ; k < sle.maxIter and residual > condition; ++k )
    {
        SLEAssistant::MatrixVectorProduct( sle.matrix, z, az );
        SLEAssistant::LLtDirect( llt, r, rLLt );
        SLEAssistant::LLtReverse( llt, rLLt, rLLt );
        b = SLEAssistant::DotProduct( rLLt, r );
        a = b / SLEAssistant::DotProduct( az, z );
        if ( std::isnan( a ) ) break;

        for ( int i = 0; i < sle.matrix.n; ++i )
        {
            sle.x[i] = sle.x[i] + a * z[i];
            r[i] = r[i] - a * az[i];
        }

        SLEAssistant::LLtDirect( llt, r, rLLt );
        SLEAssistant::LLtReverse( llt, rLLt, rLLt );
        b = SLEAssistant::DotProduct( rLLt, r ) / b;

        for ( int i = 0; i < sle.matrix.n; ++i )
            z[i] = rLLt[i] + b * z[i];

        residual = SLEAssistant::DotProduct( r, r );
    }

    std::cout << "Residual: " << sqrt( residual ) / fNorm << " k: " << k << "\n";
    return sle.x;
}

std::vector<double> MSG::SolutionWithILUConditioning( SLE sle )
{

```

```

ILU ilu( sle.matrix );
const int n = sle.matrix.n;

const double fNorm2 = SLEAssistant::DotProduct( sle.f, sle.f );
if ( fNorm2 < 1e-30 )
{
    std::cout << "Residual: 0 k: 0\n";
    return sle.x;
}
const double stopCrit = sle.eps * sle.eps * fNorm2;

std::vector<double> r( n );
std::vector<double> z( n );
std::vector<double> p( n );
std::vector<double> q( n );
std::vector<double> tmp( n );

// r = f - A*x0
SLEAssistant::MatrixVectorProduct( sle.matrix, sle.x, r );
for ( int i = 0; i < n; ++i )
    r[i] = sle.f[i] - r[i];

// z = M-1 * r, p = z
SLEAssistant::ILUDirect( ilu, r, tmp );
SLEAssistant::ILUReverse( ilu, tmp, z );
p = z;

double rz = SLEAssistant::DotProduct( r, z );
double residual2 = SLEAssistant::DotProduct( r, r );
int k = 0;

for ( ; k < sle.maxIter && residual2 > stopCrit; ++k )
{
    // q = A * p
    SLEAssistant::MatrixVectorProduct( sle.matrix, p, q );

    const double pq = SLEAssistant::DotProduct( p, q );
    if ( std::abs( pq ) < 1e-30 ) break;

    const double alpha = rz / pq;

    for ( int i = 0; i < n; ++i )
    {
        sle.x[i] += alpha * p[i];
        r[i] -= alpha * q[i];
    }

    // z_new = M-1 * r_new
    SLEAssistant::ILUDirect( ilu, r, tmp );
    SLEAssistant::ILUReverse( ilu, tmp, z );

    const double rzNew = SLEAssistant::DotProduct( r, z );
    if ( std::abs( rz ) < 1e-30 ) break;

    const double beta = rzNew / rz;
    rz = rzNew;

    for ( int i = 0; i < n; ++i )
        p[i] = z[i] + beta * p[i];

    residual2 = SLEAssistant::DotProduct( r, r );
}

std::cout << "Residual: " << std::sqrt( residual2 / fNorm2 ) << " k: " << k << "\n";
return sle.x;
}

```

6.21 Func.h

```
#pragma once
class Func
{
public:
    void SetDomain( double left, double right );
    double u( double x );
    double fc( double x );
    double fs( double x );
    double lambda( double x );
    double w( double x );
    double sigma( double x );
    double xi( double x );
    double thetaS( double x );
    double thetaC( double x );
    double beta( double x );
    double uBetaS( double x );
    double uBetaC( double x );
    double boundaryValue( int number, double x );

    // 3D тестовые функции для гармонической системы.
    double lambdaVal;
    double wVal;
    double sigmaVal;
    double xiVal;
    double lambda3D( int variant, double x, double y, double z ) const;
    double w3D( int variant, double x, double y, double z ) const;
    double sigma3D( int variant, double x, double y, double z ) const;
    double xi3D( int variant, double x, double y, double z ) const;
    double us3DExact( int variant, double x, double y, double z ) const;
    double uc3DExact( int variant, double x, double y, double z ) const;
    double LapUs( int variant, double x, double y, double z ) const;
    double LapUc( int variant, double x, double y, double z ) const;
    double fs3D( int variant, double x, double y, double z ) const;
    double fc3D( int variant, double x, double y, double z ) const;
};
```

6.22 Func.cpp

```
#include "Func.h"
#include <cmath>

namespace
{
    double gLeftZ = 0.0;
    double gRightZ = 1.0;

    double len( )
    {
        const double l = gRightZ - gLeftZ;
        return ( std::abs( l ) > 0.0 ) ? l : 1.0;
    }

    double tNorm( double z )
    {
        return ( z - gLeftZ ) / len( );
    }

    // Точное линейное решение теста:
    // us(z) = 0.2 + 0.6 * t, uc(z) = -0.4 + 0.8 * t, t в [0,1].
    double usExact( double z ) { return 0.2 + 0.6 * tNorm( z ); }
    double ucExact( double z ) { return -0.4 + 0.8 * tNorm( z ); }
    double dusExact( ) { return 0.6 / len( ); }
    double ducExact( ) { return 0.8 / len( ); }

    double lambdaCoeff( double z ) { ( void )z; return 2.0; }
    double dlambdaDz( ) { return 0.0; }
    double omegaCoeff( double z ) { ( void )z; return 5.0; }
    double sigmaCoeff( double z ) { ( void )z; return 0.7; }
```

```

double xiCoeff( double z ) { ( void )z; return 0.4; }

struct Linear3D
{
    double c0;
    double cx;
    double cy;
    double cz;
};

Linear3D us3DCoefs( int variant )
{
    return Linear3D{ -0.15, 0.35, 0.45, -0.25 };
}

Linear3D uc3DCoefs( int variant )
{
    return Linear3D{ 0.3, -0.2, 0.15, 0.5 };
}

double evalLinear3D( const Linear3D &c, double x, double y, double z )
{
    return c.c0 + c.cx * x + c.cy * y + c.cz * z;
}

double evalNonLinear3D( const Linear3D &c, double x, double y, double z )
{
    return c.cx * x*x + c.cy * y*y + c.cz * z*z;
}
}

void Func::SetDomain( double left, double right )
{
    gLeftZ = left;
    gRightZ = right;
}

double Func::u( double x )
{
    ( void )x;
    return 0.0;
}

double Func::fc( double x )
{
    const double lPrime = dlambdaDz( );
    const double duc = ducExact( );
    const double w = omegaCoeff( x );
    const double sigma = sigmaCoeff( x );
    const double xi = xiCoeff( x );

    // -(lambda * uc')' - w^2 * xi * uc + w * sigma * us = fc
    return -lPrime * duc - ( w * w ) * xi * ucExact( x ) + w * sigma * usExact( x );
}

double Func::fs( double x )
{
    const double lPrime = dlambdaDz( );
    const double dus = dusExact( );
    const double w = omegaCoeff( x );
    const double sigma = sigmaCoeff( x );
    const double xi = xiCoeff( x );

    // -(lambda * us')' - w^2 * xi * us - w * sigma * uc = fs
    return -lPrime * dus - ( w * w ) * xi * usExact( x ) - w * sigma * ucExact( x );
}

```



```

}

double Func::lambda( double x )
{
    return lambdaCoeff( x );
}

double Func::w( double x )
{
    return omegaCoeff( x );
}

double Func::sigma( double x )
{
    return sigmaCoeff( x );
}

double Func::xi( double x )
{
    return xiCoeff( x );
}

double Func::thetaS( double x )
{
    return lambdaCoeff( x ) * dusExact( );
}

double Func::thetaC( double x )
{
    return lambdaCoeff( x ) * ducExact( );
}

double Func::beta( double x )
{
    ( void )x;
    return 0.0;
}

double Func::uBetaS( double x )
{
    ( void )x;
    return 0.0;
}

double Func::uBetaC( double x )
{
    ( void )x;
    return 0.0;
}

double Func::boundaryValue( int number, double x )
{
    ( void )x;
    if ( number == 0 ) // Синусная компонента A^s
        return usExact( gLeftZ );
    if ( number == 1 ) // Косинусная компонента A^c
        return ucExact( gLeftZ );
    return 0.0;
}

double Func::lambda3D( int variant, double x, double y, double z ) const
{
    ( void )variant;
    ( void )x;
    ( void )y;
    ( void )z;
    return lambdaVal;
}

```

```

double Func::w3D( int variant, double x, double y, double z ) const
{
    ( void )variant;
    ( void )x;
    ( void )y;
    ( void )z;
    return wVal;
}

double Func::sigma3D( int variant, double x, double y, double z ) const
{
    ( void )variant;
    ( void )x;
    ( void )y;
    ( void )z;
    return sigmaVal;
}

double Func::xi3D( int variant, double x, double y, double z ) const
{
    ( void )variant;
    ( void )x;
    ( void )y;
    ( void )z;
    return xiVal;
}

double Func::us3DExact( int variant, double x, double y, double z ) const
{
    if ( variant == 2 )
        return evalLinear3D( us3DCoefs( variant ), x, y, z );
    else
        return evalNonLinear3D( us3DCoefs( variant ), x, y, z );
}

double Func::uc3DExact( int variant, double x, double y, double z ) const
{
    if ( variant == 2 )
        return evalLinear3D( uc3DCoefs( variant ), x, y, z );
    else
        return evalNonLinear3D( uc3DCoefs( variant ), x, y, z );
}

double Func::LapUs( int variant, double x, double y, double z ) const
{
    if ( variant == 2 )
        return 0;
    else
        return us3DCoefs( variant ).cx+ us3DCoefs( variant ).cy+ us3DCoefs( variant ).cz;
}

double Func::LapUc( int variant, double x, double y, double z ) const
{
    if ( variant == 2 )
        return 0;
    else
        return uc3DCoefs( variant ).cx + uc3DCoefs( variant ).cy + uc3DCoefs( variant ).cz;
}

double Func::fs3D( int variant, double x, double y, double z ) const
{
    const double lambdaVal = lambda3D( variant, x, y, z );
    const double wVal = w3D( variant, x, y, z );
    const double sigmaVal = sigma3D( variant, x, y, z );
    const double xiVal = xi3D( variant, x, y, z );
    const double lapUs = LapUs( variant, x, y, z );
    return -lambdaVal * lapUs - ( wVal * wVal ) * xiVal * us3DExact( variant, x, y, z ) -
wVal * sigmaVal * uc3DExact( variant, x, y, z );
}

```

```

double Func::fc3D( int variant, double x, double y, double z ) const
{
    const double lambdaVal = lambda3D( variant, x, y, z );
    const double wVal = w3D( variant, x, y, z );
    const double sigmaVal = sigma3D( variant, x, y, z );
    const double xiVal = xi3D( variant, x, y, z );
    const double lapUc = LapUc( variant, x, y, z );
    return -lambdaVal * lapUc - ( wVal * wVal ) * xiVal * uc3DExact( variant, x, y, z ) +
wVal * sigmaVal * us3DExact( variant, x, y, z );
}

```